StrataKV — Crash-Safe LSM-Tree Storage Engine: Build Spec

Oct 2, 2026 · @marlon

Overview

StrataKV is an embedded key-value storage engine written in C++20, built on a log-structured merge tree (LSM tree), with a Python package, crash fuzzer, property tests, and benchmarks written in Python. It is the second portfolio project after Anchor and exists to prove C++ and Python on the resume.

It tells the same story as Anchor from a different layer. Anchor makes agent workflows survive crashes; StrataKV makes data survive crashes. Together they say: I build systems that do not lose things.

What it is, in one sentence: a tiny RocksDB. You `put`, `get`, and `delete` keys, every acknowledged write survives `kill -9`, and reads stay fast as data grows past memory.

Success criteria (the project is done when all of these are true):

- [ ] Zero acknowledged writes lost across at least 1,000 randomized `SIGKILL` crashes, including crashes during flush and compaction
- [ ] Hypothesis stateful tests pass at least 10,000 random operation sequences against a Python dict model, including reopen-after-close
- [ ] Clean under AddressSanitizer, UndefinedBehaviorSanitizer, and ThreadSanitizer
- [ ] `pip install .` produces an importable `stratakv` module on Linux and macOS
- [ ] Benchmarks against SQLite are published in the README, with charts and the exact commands used
- [ ] Every number on the resume comes from `docs/NUMBERS.md`, which records the command, machine, and date that produced it

Out of scope: networking, replication, transactions across keys beyond atomic write batches, range-scan iterators (stretch goal only), Windows support.

How this doc is organized:

Tab	What it holds	Read it when
This tab	The spec, design decisions, ten annotated code blocks, and the final exam	Before each phase
C++ primer	Every C++ feature used in StrataKV, explained next to Python, plus pybind11 and CMake basics	Before reading any code
Glossary	Every storage, systems, and concurrency term in plain English	Whenever a word is unfamiliar
Checkpoint quizzes	Five questions per phase with answer keys	After each phase

Instructions for Claude Code

Build StrataKV phase by phase, in order, and stop at the end of each phase for review. Do not start a phase until the previous phase's acceptance criteria all pass.

Working rules:

1. Read this entire document before writing code. The design decisions section is binding; if you think one is wrong, write the objection in `docs/DEBRIEF.md` and ask before deviating.
2. Write tests in the same phase as the code they test. A phase is not done until its tests pass under the `asan` and `tsan` build presets.
3. Never invent a performance number. Every number in the README or resume must come from a command that was actually run, recorded in `docs/NUMBERS.md` as: metric, value, exact command, machine (CPU, OS, disk), date.
4. Keep the code readable for a learner. Prefer clear names over clever tricks, and put a short comment above every function that touches durability (fsync, rename, WAL replay) explaining why the order of operations matters.
5. Commit at least once per phase with a conventional message (`feat(wal): ...`, `test(fuzz): ...`). Small, reviewable commits.
6. No dependencies beyond those listed in the tech stack section without asking.

End-of-phase debrief (required). Append a section to `docs/DEBRIEF.md` for each phase containing:

- What was built, in plain English, in five sentences or fewer
- The three to five most important files and what each one does
- Every decision made that this spec did not dictate, with the alternative considered and why it lost
- Any bug that took more than one attempt to fix, what caused it, and how it was found
- Five quiz questions with answers, written so Marlon can test himself on that phase
- Measured numbers, if the phase produced any, copied into `docs/NUMBERS.md`

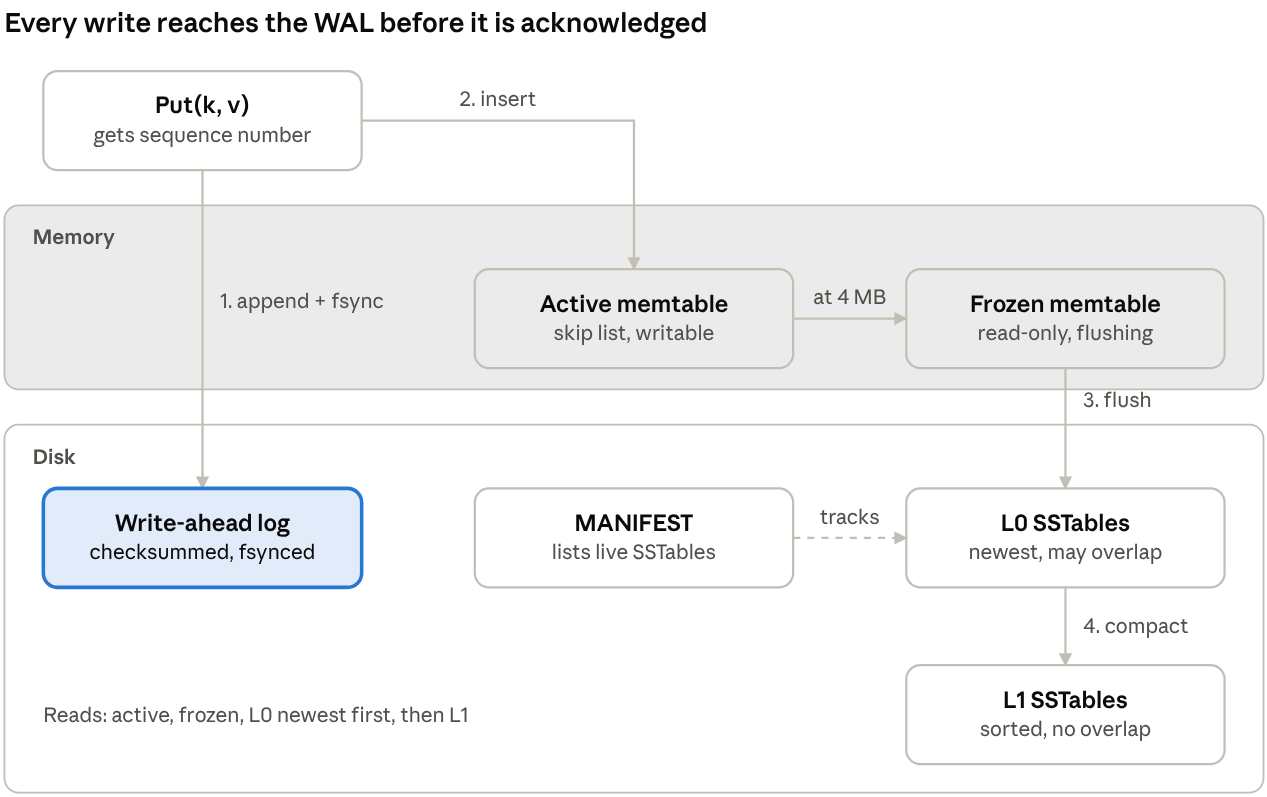
Owner's note. Marlon will review each debrief, run the tests himself, and must be able to explain every line in the learning guide below before StrataKV goes on a resume.

Learning protocol (required after every phase, alongside the debrief). Marlon is strong in Python and new to C++ and storage engines, so write for that reader. Create `docs/learning/PN-<name>.md` containing:

- **Plain-English summary:** what was built and why, under 150 words, every term defined or linked to the Glossary tab
- **Data flow:** a step-by-step trace of one `Put` or `Get` through the code from this phase, naming files and functions
- **Annotated real code:** the three to five most important functions from the actual implementation, each followed by a line-by-line explanation and, where it helps, the closest Python equivalent; compare against the matching snippet in the learning guide below and explain any differences
- **New C++ features:** any language feature not yet in the C++ primer tab, explained the same way; add it to `docs/CPP_NOTES.md`
- **New terms:** added to `docs/GLOSSARY.md`
- **Struggles:** bugs that took more than one attempt, with cause, how it was found, and the lesson. These become interview stories.
- **Checkpoint gate:** do not start the next phase until Marlon reports passing that phase's quiz in the Checkpoint quizzes tab at 80% or better

Architecture and on-disk formats

Writes go to a write-ahead log first, then into memory; memory is periodically flushed to immutable sorted files on disk; a background thread merges those files. Reads check newest data first and stop at the first match.



StrataKV write path · memory and disk

The WAL is the only thing a write must reach before it is acknowledged; everything after step 2 happens in the background.

Write path: `Put(k, v)` → assign sequence number → append record to WAL → `fsync` (per sync policy) → insert into active memtable → acknowledge. When the memtable passes 4 MB it becomes immutable, a new WAL and memtable start, and a background thread flushes the immutable memtable to a new L0 SSTable. The old WAL is deleted only after the MANIFEST records the new SSTable.

Read path: active memtable → immutable memtable → L0 files newest to oldest (they may overlap) → the single L1 file whose key range covers the key (L1 files never overlap). Each SSTable checks its bloom filter before touching disk.

Compaction: when L0 reaches 4 files, merge all L0 files plus the overlapping L1 files into new L1 files, dropping overwritten versions and dropping tombstones (deletes) because L1 is the bottom level.

Internal key. Every stored key is `user_key | seq (7 bytes) | type (1 byte, PUT or DELETE)`. Ordering is user key ascending, then sequence number descending, so the newest version of a key is always found first.

WAL record format (little-endian):

```
+-----------+-----------+-----------+----------------------------+
| crc32c(4) | length(4) | type(1)   | payload (length bytes)     |
+-----------+-----------+-----------+----------------------------+
payload = seq(8) | count(4) | { op(1) | klen(varint) | key | vlen(varint) |
value } * count
crc covers type + payload. One record = one atomic WriteBatch.
```

On recovery, records are replayed until the first record that is incomplete or fails its checksum; the file is truncated there. A torn final record is normal after a crash and is never an error, because it was never acknowledged.

SSTable format:

```
[data block 0][data block 1]...[data block N]   ~4 KB each, sorted internal
keys
[filter block]                                  bloom filter over all user
keys, 10 bits/key
[index block]                                   last key of each data block -
> (offset, size)
[footer, 48 bytes]                              filter (offset,size) | index
(offset,size) | magic | crc32c
```

Data blocks use a plain `klen | key | vlen | value` encoding. Prefix compression and restart points are a stretch goal, not required.

MANIFEST and CURRENT. The MANIFEST is an append-only log of version edits (file added to level, file removed, next file number, last sequence number), using the same record framing as the WAL. `CURRENT` holds the name of the active MANIFEST and is replaced atomically with write-temp, `fsync`, `rename`, `fsync` directory. On open: read CURRENT → replay MANIFEST → replay any WAL newer than the last flushed sequence number.

Directory layout of a database:

```
mydb/
  CURRENT              -> "MANIFEST-000005"
  MANIFEST-000005
  000007.log           active WAL
  000004.sst           L0
  000006.sst           L1
  LOCK                 flock() so two processes cannot open the same DB
```

Tech stack and repo layout

C++20 for the engine, Python 3.11+ for everything around it, CMake to build both, GitHub Actions to test on Linux and macOS.

Layer	Choice	Purpose
Engine language	C++20 (GCC 12+, Clang 15+)	`std::span`, `std::shared_mutex`, `std::atomic`, concepts where they help readability
Build	CMake 3.24+ with presets, `FetchContent`	Presets: `debug`, `release`, `asan` (ASan + UBSan), `tsan`
Checksums	CRC32C, hand-written table implementation	No dependency; about 40 lines
C++ tests	GoogleTest	Unit tests per component
C++ microbenchmarks	Google Benchmark (optional, P6)	Skip list and bloom filter microbenchmarks
Python bindings	pybind11	`import stratakv`
Python packaging	scikit-build-core, `pyproject.toml`	`pip install .` builds the extension via CMake
Python tests	pytest, Hypothesis	Stateful model-based tests and the crash fuzzer
Benchmarks and charts	Python, `sqlite3` (stdlib), matplotlib	StrataKV vs SQLite
Formatting and linting	clang-format, clang-tidy, ruff	Enforced in CI

Layer	Choice	Purpose
CI	GitHub Actions, matrix: ubuntu-latest, macos-latest	Build, all presets' tests, Python tests, short fuzz run

Repo layout:

```
stratakv/
  CMakeLists.txt  CMakePresets.json  pyproject.toml  README.md
  include/stratakv/
    db.h            public API: DB::Open, Put, Get, Delete,
Write(WriteBatch), Close
    options.h       Options (memtable size, sync policy), ReadOptions,
WriteOptions
    status.h        Status: Ok, NotFound, Corruption, IOError,
InvalidArgument
    write_batch.h
  src/
    db_impl.{h,cc}        orchestration, write queue, background thread
    wal_writer.{h,cc}     wal_reader.{h,cc}
    memtable.{h,cc}       skiplist.h
    internal_key.{h,cc}   comparator, encoding helpers, varint
    sstable_builder.{h,cc}  sstable_reader.{h,cc}  bloom.{h,cc}
    version.{h,cc}        Version, VersionSet, VersionEdit, MANIFEST
    compaction.{h,cc}     merging_iterator.{h,cc}
    env.{h,cc}            file I/O, fsync (F_FULLFSYNC on macOS), rename,
flock
    crash_point.{h,cc}    test-only crash injection controlled by env var
    crc32c.{h,cc}
  tests/cpp/              GoogleTest suites
  python/
    bindings.cc           pybind11 module
    stratakv/__init__.py  typed wrapper, context manager, MutableMapping
interface
  tests/py/
    test_basic.py  test_stateful.py (Hypothesis)  crash_fuzz.py
  bench/
    bench.py  plot.py  results/
  docs/
    DEBRIEF.md  NUMBERS.md  DESIGN.md
  .github/workflows/ci.yml
```

Build phases

Seven phases, each ending in passing tests and a debrief. Correctness comes before speed: crash safety is proven in P1 and re-proven after every phase that touches disk.

P0 — Scaffold and CI

- CMake project with the four presets, GoogleTest via FetchContent, clang-format and clang-tidy configs
- `pyproject.toml` with scikit-build-core and pybind11; a stub module exposing `stratakv.version()`
- GitHub Actions matrix (Ubuntu, macOS) running build, C++ tests, and pytest
- `Status`, `Options`, `Env` (file open, append, `fsync`, `rename`, directory `fsync`, `flock`), CRC32C with known-answer tests
- Empty `docs/DEBRIEF.md`, `docs/NUMBERS.md`, `docs/DESIGN.md`

Acceptance: CI is green on both OSes; `pip install .` then `python -c "import stratakv; print(stratakv.version())"` works; CRC32C matches the standard test vector for `"123456789"` (`0xE3069283`).

P1 — WAL, memtable, and crash recovery

- `WriteBatch`, internal key encoding, comparator, varint helpers
- Skip list memtable: single writer, readers without locks using acquire/release atomics
- WAL writer and reader with the record format above; torn-tail truncation on recovery
- `DB::Open` (with `LOCK` file), `Put`, `Get`, `Delete`, `Write(batch)`, `Close`
- Sync policy option: `kSyncEveryWrite` (default) and `kNoSync`
- Crash-point framework: `STRATAKV_CRASH_AT=<name>` makes the process `_exit(1)` at a named point

Acceptance: unit tests for every component; a test writes 10,000 keys, simulates a torn final record by truncating the WAL mid-record, reopens, and recovers every complete record; corrupting one byte in the middle record is detected as a checksum failure.

P2 — SSTables, flush, and the read path

- `SSTableBuilder` and `SSTableReader` with data blocks, index block, bloom filter, footer

- Bloom filter: 10 bits per key, double hashing, measured false-positive rate test (expect about 1%)
- Memtable rotation at 4 MB, background flush thread, old WAL deleted only after the flush is durable
- Read path across memtable, immutable memtable, and L0 files

Acceptance: write 1,000,000 keys (well past one memtable), reopen, read all back correctly; bloom filter false-positive rate is between 0.5% and 2% on 100,000 absent keys; `STRATAKV_CRASH_AT=flush_after_sst_write` and `flush_before_wal_delete` both recover with no lost or duplicated data.

P3 — MANIFEST, versions, and compaction

- `VersionEdit`, `VersionSet`, MANIFEST log, atomic `CURRENT` replacement
- `Version` objects held by `std::shared_ptr`; readers take a snapshot so files are never deleted under them; obsolete files deleted when no version references them
- Merging iterator (k-way merge with a min-heap) that keeps only the newest version of each user key and drops tombstones at L1
- L0 to L1 compaction triggered at 4 L0 files; L1 split into files of about 8 MB

Acceptance: after writing 5,000,000 random keys with 30% overwrites and 10% deletes, the database matches a reference `std::map`; disk usage after compaction is within 1.5× of live data size; crash points `compaction_after_output` and `compaction_before_manifest` both recover correctly.

P4 — Concurrency and group commit

- Writer queue with leader-follower group commit: the leader merges queued batches into one WAL record and issues one `fsync` for all of them
- Concurrent readers during writes, flushes, and compactions
- Stress test: 8 writer threads and 8 reader threads for 60 seconds

Acceptance: the stress test passes under the `tsan` preset with zero reports; group commit with 8 writers achieves at least 3× the throughput of 8 writers with group commit disabled, measured and recorded in `NUMBERS.md`.

P5 — Python package, property tests, crash fuzzer

- pybind11 module: `DB` class with `open`, `get`, `put`, `delete`, `write_batch`, `close`, `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, context manager; releases the GIL during engine calls; maps `Status` codes to Python exceptions
- Hypothesis `RuleBasedStateMachine` comparing StrataKV to a dict, with rules for put, delete, get, batch, and close-then-reopen
- `crash_fuzz.py`: a child process writes random keys and prints each acknowledged key to a pipe; the parent kills it with `SIGKILL` at a random moment, optionally also arming a random crash point; then reopens and verifies every acknowledged write is present

Acceptance: stateful tests pass 10,000 examples; the fuzzer completes 1,000 iterations with zero lost acknowledged writes and zero corruption errors; CI runs a 50-iteration fuzz on every push.

P6 — Benchmarks, README, and polish

- `bench/bench.py` workloads: sequential fill, random fill, random read (hits), random read (misses, to show bloom filters), overwrite-heavy; value sizes 100 bytes and 1 KB
- Compare against SQLite in WAL mode at both `synchronous=FULL` and `NORMAL`, matched to StrataKV's sync policy for fairness
- Measure write amplification (bytes written to disk divided by bytes written by the user)
- `bench/plot.py` produces charts into `bench/results/`
- README: what it is, architecture diagram, how to install, crash-safety guarantees and how they are tested, benchmark results with exact commands, design decisions, known limitations

Acceptance: every README number is in `NUMBERS.md` with its command; a fresh clone can reproduce the benchmarks with one command.

Stretch (only after P6): range-scan iterators exposed to Python, prefix compression in data blocks, a block cache, a Google Benchmark suite.

Design decisions and trade-offs

Every major choice below was made to maximize what an interviewer can probe and what can be finished reliably, not to maximize raw performance. Know the "what it costs" column as well as the "why" column; interviewers ask about costs.

Why this project instead of another C++ project

Option	Why it lost
StrataKV (chosen)	Matches the Anchor story (durability, crash recovery), covers C++ and Python in one repo, and maps directly to classic system-design questions
Thread pool or lock-free queue	Too small to stand alone; every candidate has one. Its best ideas (atomics, memory ordering) live inside StrataKV's skip list and group commit anyway
HTTP server or Redis clone	Very common on new-grad resumes, so it does not stand out; networking would overlap with Anchor instead of adding something new
Order book matching engine	Strong C++ signal, but reads as a quant project, and quant is not the current target
Raft consensus	Impressive but too large for the time available, and Go is the more natural language for it, which Anchor already covers
Ray tracer or game engine	Good C++ but unrelated to backend and infrastructure roles

Engine design choices

Decision	Chose	Main alternative	Why it won	What it costs
Storage structure	LSM tree	B-tree (SQLite, InnoDB)	Writes are sequential appends; immutable files make crash safety simple; LSM vs B-tree is a standard interview question	Reads may check several files (read amplification); compaction rewrites data (write amplification)
Memtable	Custom skip list	`std::map` behind a `shared_mutex`	Readers need no lock, which shows real atomics and memory-ordering skill; same choice as LevelDB and RocksDB	Harder to get right; must be tested under TSan
Compaction	Two-level leveled (L0, L1)	Size-tiered (Cassandra) or full multi-level leveled	Leveled keeps reads and disk usage low; two levels keep it buildable while still teaching the concept	Higher write amplification than size-tiered; very large datasets would need more levels
Durability	Sync per write by default, plus group commit	Periodic sync every N ms	Every acknowledged write survives a crash, which is the project's core claim; group commit recovers most of the throughput	Single-threaded writes are slower than periodic sync

Decision	Chose	Main alternative	Why it won	What it costs
Error handling	`Status` return values	C++ exceptions	No hidden control flow on the hot path; matches LevelDB and RocksDB; converts cleanly to Python exceptions at the binding layer	More verbose call sites; every result must be checked
Python bindings	pybind11	nanobind, Cython, ctypes	Most widely used and documented, so it is the one interviewers recognize	Slightly slower and larger than nanobind
Correctness testing	Hypothesis stateful tests in Python	RapidCheck property tests in C++	Covers Python meaningfully; model-based testing against a dict is far faster to write in Python	Tests run through the binding layer, so a pure C++ bug can show up one layer removed
Build	CMake with FetchContent	Bazel, or Conan/vcpkg	Industry default for C++; scikit-build-core uses it directly for the Python build	CMake's syntax is clumsy

Alternative approach worth knowing: a B-tree engine

If asked "how else could you have built this?", the strongest answer is a copy-on-write B-tree, as in LMDB and BoltDB. Pages are never modified in place; a write copies the path from leaf to root and atomically swaps the root pointer.

- **Pros:** reads touch one structure, so lookups are fast and predictable; no compaction; built-in snapshots.

- **Cons:** every write copies several pages, so random-write throughput is lower; free-page management is tricky; a single writer at a time.
- **When to pick it:** read-heavy workloads, like a configuration store. Pick an LSM tree for write-heavy workloads, like logs, metrics, and event journals such as Anchor's.

Learning guide: the code that matters most

These ten snippets carry roughly 80% of what interviewers will ask about. They are simplified teaching versions; Claude Code's real implementation will differ in details, so compare each one against the real file after the matching phase.

1. WAL append and recovery (P1) — durability

```
// Append one WriteBatch as one record. It is acknowledged only after Sync()
returns.
Status WalWriter::AddRecord(std::span<const uint8_t> payload) {
  uint8_t header[9];
  const uint8_t type = kFullRecord;
  uint32_t crc = crc32c::Extend(crc32c::Value(&type, 1), payload.data(),
payload.size());
  EncodeFixed32(header, crc);
  EncodeFixed32(header + 4, static_cast<uint32_t>(payload.size()));
  header[8] = type;
  if (auto s = file_->Append({header, 9}); !s.ok()) return s;
  if (auto s = file_->Append(payload); !s.ok()) return s;
  return sync_every_write_ ? file_->Sync() : Status::OK();
}

// Replay until the first incomplete or corrupt record, then cut the file
there.
Status WalReader::Replay(const std::function<void(std::span<const uint8_t>)>&
apply) {
  uint64_t offset = 0;
  while (true) {
    uint8_t header[9];
    if (!file_->ReadAt(offset, header, 9)) break;                // torn
header
    uint32_t crc = DecodeFixed32(header);
    uint32_t len = DecodeFixed32(header + 4);
    if (len > kMaxRecordSize) break;                             // garbage
length
```

```
    std::vector<uint8_t> buf(1 + len);
    buf[0] = header[8];
    if (!file_->ReadAt(offset + 9, buf.data() + 1, len)) break;   // torn
payload
    if (crc32c::Value(buf.data(), buf.size()) != crc) break;      // torn or
corrupt
    apply({buf.data() + 1, len});
    offset += 9 + len;
  }
  return file_->Truncate(offset);
}
```

What to understand: truncating the tail is safe because a record that was not fully written was never acknowledged, so no caller was promised it. The checksum is what tells "half-written" apart from "complete." The real reader should also check whether valid records exist *after* a bad one; if so, that is mid-file corruption and must be reported as `Status::Corruption`, not silently truncated.

2. Atomic file replacement (P3) — the four-step durability dance

```
// After a crash, CURRENT holds either the old name or the new one, never
half of either.
Status SetCurrentFile(Env& env, const std::string& dir, const std::string&
manifest) {
  const std::string tmp = dir + "/CURRENT.tmp";
  if (auto s = env.WriteFile(tmp, manifest + "\n"); !s.ok()) return s;
  if (auto s = env.SyncFile(tmp); !s.ok()) return s;               // 1.
contents on disk
  if (auto s = env.Rename(tmp, dir + "/CURRENT"); !s.ok()) return s;  // 2.
atomic name swap
  return env.SyncDir(dir);                                          // 3. the
rename on disk
}
```

What to understand: skip step 1 and a crash can leave CURRENT pointing at an empty file, because the rename reached disk before the data did. Skip step 3 and the rename itself can vanish. On macOS, plain `fsync` does not force the drive's cache to flush, so `Env::SyncFile` uses `fcntl(fd, F_FULLFSYNC)`. That detail is a great interview moment.

3. Skip list insert with memory ordering (P1) — lock-free reads

```
// One writer at a time (it holds the write lock); any number of readers with
no lock.
void SkipList::Insert(std::string key, std::string value) {
  Node* prev[kMaxHeight];
  Node* x = head_;
  const int cur = max_height_.load(std::memory_order_relaxed);
  for (int level = cur - 1; level >= 0; --level) {
    Node* next = x->next[level].load(std::memory_order_acquire);
    while (next != nullptr && cmp_(next->key, key) < 0) {
      x = next;
      next = x->next[level].load(std::memory_order_acquire);
    }
    prev[level] = x;
  }
  const int height = RandomHeight();          // 1 + geometric(p = 1/4),
capped
  if (height > cur) {
    for (int l = cur; l < height; ++l) prev[l] = head_;
    max_height_.store(height, std::memory_order_relaxed);
  }
  Node* node = arena_.New<Node>(std::move(key), std::move(value), height);
  for (int l = 0; l < height; ++l) {
    // Set the new node's own pointer first; no reader can see the node yet.
    node->next[l].store(prev[l]->next[l].load(std::memory_order_relaxed),
                        std::memory_order_relaxed);
    // Publish. Release means any reader that sees `node` also sees its key,
value, next.
    prev[l]->next[l].store(node, std::memory_order_release);
  }
}
```

What to understand: the release store pairs with the readers' acquire loads. Without that pairing, a reader on another core could see the pointer to `node` before it sees the node's key, and read garbage. Nodes are never freed individually; the whole arena is freed when the memtable is discarded, which is why readers never touch freed memory.

4. Internal key ordering (P1) — how "newest wins"

```
// User key ascending, then (sequence << 8 | type) DESCENDING: newest version
sorts first.
int InternalKeyComparator::Compare(std::string_view a, std::string_view b)
const {
  std::string_view ua = a.substr(0, a.size() - 8), ub = b.substr(0, b.size()
- 8);
  if (int c = ua.compare(ub); c != 0) return c;
  uint64_t ta = DecodeFixed64(a.data() + a.size() - 8);
  uint64_t tb = DecodeFixed64(b.data() + b.size() - 8);
  return ta > tb ? -1 : (ta < tb ? 1 : 0);
}
```

What to understand: a `Get` seeks to `(key, current_seq)` and takes the first entry with a matching user key. If that entry is a DELETE, the answer is "not found," even if older PUTs exist below it. Deletes are just newer entries, called tombstones.

5. Bloom filter (P2) — skipping disk reads

```
// k probes from one hash via double hashing: h_i = h + i * delta.
// 10 bits per key with k = 7 gives about a 0.8% false-positive rate.
void BloomBuilder::Add(std::string_view key) {
  uint32_t h = Hash(key);
  const uint32_t delta = (h >> 17) | (h << 15);
  for (int i = 0; i < k_; ++i) {
    const uint32_t bit = h % num_bits_;
    bits_[bit / 8] |= static_cast<uint8_t>(1u << (bit % 8));
    h += delta;
  }
}
// MayContain runs the same loop and returns false at the first unset bit.
// "false" is certain; "true" only means "maybe, go read the block".
```

What to understand: the optimal number of probes is k = ln 2 × bits per key, about 7 for 10 bits. Bloom filters only help lookups for keys that are absent from a file, which is most lookups in an LSM tree because a key lives in one file but the read path may check many.

6. K-way merge for compaction (P3) — dropping old versions

```
void Compact(std::vector<std::unique_ptr<Iterator>>& inputs, SSTableBuilder&
out, bool bottom) {
  auto greater = [&](Iterator* a, Iterator* b) { return icmp_.Compare(a-
>key(), b->key()) > 0; };
  std::priority_queue<Iterator*, std::vector<Iterator*>, decltype(greater)>
heap(greater);
  for (auto& it : inputs) { it->SeekToFirst(); if (it->Valid())
heap.push(it.get()); }

  std::string last_user_key;
  bool have_last = false;
  while (!heap.empty()) {
    Iterator* it = heap.top(); heap.pop();
    std::string_view ikey = it->key();
    std::string_view ukey = UserKey(ikey);
    const bool shadowed = have_last && ukey == last_user_key;   // newer
version already handled
    if (!shadowed) {
      last_user_key.assign(ukey.data(), ukey.size());
      have_last = true;
      const bool tombstone = TypeOf(ikey) == kTypeDeletion;
      if (!(tombstone && bottom)) out.Add(ikey, it->value());
    }
    it->Next();
    if (it->Valid()) heap.push(it);
  }
}
```

What to understand: because of the ordering in snippet 4, the first entry seen for each user key is the newest; everything after it is garbage. Tombstones can only be dropped at the bottom level. Drop one higher up and an older PUT in a lower level would come back to life.

7. Version snapshots (P3) — reading while files are deleted

```
Status DBImpl::Get(std::string_view key, std::string* value) {
  std::shared_ptr<MemTable> mem, imm;
  std::shared_ptr<const Version> v;
  uint64_t snapshot;
  {
    std::lock_guard lock(mu_);   // tiny critical section: copy pointers,
nothing else
    mem = mem_; imm = imm_; v = current_; snapshot = last_sequence_;
  }
  // Search mem, then imm, then v's files, with no lock held.
  // Compaction can install a new Version meanwhile; our `v` keeps the old
files alive.
  // When the last shared_ptr to an old Version dies, its unused files become
deletable.
  ...
}
```

What to understand: this is reference counting used as a concurrency tool. It avoids holding a lock across disk reads, which would block every writer.

8. Group commit (P4) — one fsync for many writers

```
Status DBImpl::Write(WriteBatch* batch) {
  Writer w{batch};
  std::unique_lock lock(mu_);
  writers_.push_back(&w);
  w.cv.wait(lock, [&] { return w.done || &w == writers_.front(); });
  if (w.done) return w.status;                  // a leader already wrote our
batch

  auto [group, last] = BuildBatchGroup();       // leader merges all queued
batches
  const uint64_t seq = last_sequence_ + 1;
  group.SetSequence(seq);
  lock.unlock();                                // new writers can queue
during I/O
  Status s = wal_->AddRecord(group.Contents());  // ONE write, ONE fsync for
everyone
  if (s.ok()) s = InsertInto(group, mem_.get());
  lock.lock();
  last_sequence_ = seq + group.Count() - 1;
```

```
  while (true) {                                  // wake everyone in the group
    Writer* f = writers_.front(); writers_.pop_front();
    if (f != &w) { f->status = s; f->done = true; f->cv.notify_one(); }
    if (f == last) break;
  }
  if (!writers_.empty()) writers_.front()->cv.notify_one();  // next leader
  return s;
}
```

What to understand: `fsync` costs about the same whether it flushes 100 bytes or 100 KB, so batching amortizes it. While the leader does I/O, new writers pile up in the queue and become the next group, so throughput rises with load. Postgres, MySQL, and Kafka all use this idea.

9. pybind11 binding (P5) — crossing languages safely

```
namespace py = pybind11;

PYBIND11_MODULE(_stratakv, m) {
  py::register_exception<CorruptionError>(m, "CorruptionError");
  py::class_<stratakv::DB>(m, "DB")
    .def("get", [](stratakv::DB& db, py::bytes key) -> py::object {
        std::string k = key;   // copy out of Python-owned memory while
holding the GIL
        std::string v;
        stratakv::Status s;
        {
          py::gil_scoped_release no_gil;   // let other Python threads run
during disk I/O
          s = db.Get(k, &v);
        }
        if (s.IsNotFound()) return py::none();
        ThrowIfError(s);                  // Status -> Python exception
        return py::bytes(v);
      }, py::arg("key"));
}
```

What to understand: the GIL (global interpreter lock) lets only one thread run Python at a time. Releasing it during C++ work lets Python threads actually run in parallel. You must copy the key *before* releasing, because Python objects can only be touched while holding the GIL.

10. Model-based testing and the crash fuzzer (P5) — proving correctness

```
class KVMachine(RuleBasedStateMachine):
    keys = st.binary(min_size=1, max_size=16)

    def __init__(self):
        super().__init__()
        self.path = tempfile.mkdtemp()
        self.db = stratakv.DB.open(self.path)
        self.model: dict[bytes, bytes] = {}

    @rule(k=keys, v=st.binary(max_size=256))
    def put(self, k, v):
        self.db.put(k, v)
        self.model[k] = v

    @rule(k=keys)
    def delete(self, k):
        self.db.delete(k)
        self.model.pop(k, None)

    @rule(k=keys)
    def get(self, k):
        assert self.db.get(k) == self.model.get(k)

    @rule()
    def reopen(self):              # every acknowledged write must survive
close/open
        self.db.close()
        self.db = stratakv.DB.open(self.path)

    def teardown(self):
        self.db.close()
        shutil.rmtree(self.path)


def crash_iteration(db_dir: str, rng: random.Random) -> None:
    child = subprocess.Popen([sys.executable, "-m", "tests.py.fuzz_writer",
db_dir],
                             stdout=subprocess.PIPE, text=True)
    time.sleep(rng.uniform(0.05, 2.0))
    child.kill()                   # SIGKILL: no cleanup, no destructors, no
flush
```

```
    acked = dict(line.split() for line in child.stdout.read().splitlines())
    child.wait()
    with stratakv.DB.open(db_dir) as db:
        for k, v in acked.items():
            assert db.get(k.encode()) == v.encode(), f"lost acknowledged
write {k}"
```

What to understand: Hypothesis generates random sequences of operations and, when one fails, shrinks it to the smallest failing sequence, often three or four steps. In the fuzzer, the child prints a key only *after* `put` returns, so the printed set is exactly what was promised. Each key is written once per iteration; otherwise a later, unacknowledged write of the same key could legitimately be on disk and cause a false failure.

Final exam

Twenty questions, roughly in the order an interviewer would escalate. Answer each out loud before reading the key. Being able to answer all twenty cold, plus three prepared stories (hardest bug, a decision you'd change, the project's limits) drawn from the Struggles sections of `docs/learning/`, is the bar for putting StrataKV on the resume. Shorter checkpoint quizzes for each phase are in the Checkpoint quizzes tab.

Fundamentals

1. Walk through what happens, step by step, when someone calls `Put("a", "1")`.
2. Walk through a `Get` for a key that was deleted an hour ago.
3. Why does an LSM tree write faster than a B-tree for random keys?
4. What are read, write, and space amplification, and which one does compaction trade against which?
5. Why can L0 files overlap but L1 files cannot?

Durability

6. A crash happens halfway through writing a WAL record. What happens on restart, and why is that correct?
7. Why do you `fsync` the directory after `rename`?
8. Why is the old WAL deleted only after the MANIFEST records the new SSTable? What breaks if you reverse the order?
9. What does "acknowledged" mean in your guarantee, and why does the fuzzer only check acknowledged writes?

10. Why does macOS need `F_FULLFSYNC`?

Concurrency

11. How can readers search the skip list without a lock while a writer inserts?
12. What would break if the publishing store in the skip list used `memory_order_relaxed`?
13. How do you delete SSTable files that a compaction made obsolete without breaking an in-flight read?
14. Explain group commit. Why does throughput go up as more threads write?

Design trade-offs

15. Why can tombstones only be dropped at the bottom level?
16. Why 10 bits per key in the bloom filter, and what happens at 5 or 20?
17. When would you choose a B-tree engine instead of this design?
18. How would you add range scans? What makes them harder than point lookups in an LSM tree?

Python and testing

19. Why release the GIL in the bindings, and why must the key be copied first?
20. What does Hypothesis's shrinking do, and why does it matter for a storage engine?

Answer key

1. Assign the next sequence number, encode a WAL record, append it, `fsync`, insert `(a, seq, PUT) → 1` into the memtable, return OK. Flush and compaction happen later in the background.
2. Check memtable, immutable memtable, L0 newest to oldest, then L1. The first entry found for that key is the tombstone (the newest version), so return NotFound without looking further.
3. Every write is a sequential append to the WAL plus an in-memory insert. A B-tree must find and modify the right page, which means random I/O.
4. Read amplification: files checked per lookup. Write amplification: bytes written to disk per byte the user writes. Space amplification: disk used versus live data. Compaction spends write amplification to reduce read and space amplification.
5. Each L0 file is a flushed memtable covering whatever keys were written, so ranges overlap. L1 is produced by compaction, which writes one sorted, non-overlapping run.
6. Replay stops at the incomplete record and truncates it. Correct because that write's `fsync` never returned, so it was never acknowledged.

7. A rename changes the directory's contents, and the directory entry is its own data that may still sit in cache. Without the directory `fsync`, the rename can be lost after a crash.
8. The WAL is the only durable copy until the MANIFEST says the SSTable exists. Delete it first and a crash in between loses that data.
9. `Put` returned OK to the caller. Unacknowledged writes may or may not survive, and both outcomes are correct, so only acknowledged writes can be checked.
10. On macOS, `fsync` sends data to the drive but does not force the drive to flush its own write cache. `F_FULLFSYNC` does, which is what real durability requires.
11. The writer fully builds the new node, then publishes it with a single release store. Readers use acquire loads, so seeing the pointer guarantees seeing the node's contents. Nodes are never freed while the memtable is alive.
12. A reader could see the pointer to the new node before its key and value are visible, and read uninitialized memory. It might only fail rarely on x86 but more often on ARM, such as Apple Silicon.
13. Each read holds a `shared_ptr` to the Version it started with. A file is deleted only once no live Version references it.
14. Writers queue; the first becomes leader, merges all queued batches, and does one write and one `fsync` for all of them. Since `fsync` cost barely depends on size, more writers means more batches per `fsync`.
15. A tombstone hides older PUTs in lower levels. Drop it above the bottom and those older values reappear.
16. Ten bits with 7 probes gives about 1% false positives. Five bits gives about 9%, wasting many disk reads; 20 bits gives about 0.01% but doubles filter memory, which is diminishing returns.
17. Read-heavy workloads that need fast, predictable lookups, or when you need cheap snapshots, like LMDB's copy-on-write design.
18. A merging iterator across the memtables and every relevant file, skipping shadowed versions and tombstones. Harder because every level must be consulted, while a point lookup can stop at the first match.
19. So other Python threads can run during disk I/O. Python objects may only be touched while holding the GIL, so the key must be copied into C++ memory first.
20. When a random test fails, Hypothesis searches for the smallest operation sequence that still fails. A 3-step reproduction of a recovery bug is debuggable; a 4,000-step one is not.

Resume entry and AI tooling

Add StrataKV as the second project, below Anchor, and fold Premier Edge into the Premier Equity experience entry to keep one page. Fill every blank only from `docs/NUMBERS.md`; if a number was never measured, delete that clause.

StrataKV — Crash-Safe LSM-Tree Storage Engine | C++20, CMake, pybind11, Python, GoogleTest, Hypothesis

- Built an embedded key-value storage engine on a log-structured merge tree: checksummed write-ahead log, lock-free skip-list memtable, SSTables with bloom filters, and background leveled compaction
- Proved crash safety with a fuzzer that `SIGKILL`s the process at random points, including mid-flush and mid-compaction: zero acknowledged writes lost across ___ crashes
- Implemented leader-follower group commit, raising 8-thread durable write throughput ___× by sharing one `fsync` across batched writers; ThreadSanitizer-clean under concurrent reads, writes, and compaction
- Shipped a pybind11 Python package validated by Hypothesis stateful tests over ___ random operation sequences; benchmarked ___ writes/sec vs SQLite's ___ at equal durability settings

Language coverage after StrataKV

Language	Shown by
Go	Anchor runtime and workers
TypeScript / JavaScript	Anchor trace viewer (Next.js), Coog Zoo (React, Node, Express), Premier Edge
C++	StrataKV engine
Python	StrataKV bindings, fuzzer, property tests, benchmarks

Where LLM APIs and Claude Code belong on the resume

List each tool where it is actually true, so every claim survives a follow-up question.

- **LLM APIs:** Anchor calls model APIs in production code, so name the specific providers it calls in Anchor's tech line, for example "Anthropic Claude API". Premier Edge's LLM agent counts too, so name its provider there.
- **Claude Code:** add it to the skills section as a development tool, for example `Developer tools: Claude Code (AI-assisted development), Git, Docker, GitHub Actions`. Many companies now ask about AI tool use, and fluency with it is a plus.
- **Do not** list LLM APIs on StrataKV. It does not call one, and forcing one in would be the kind of claim an interviewer can unravel in one question. Anchor already proves LLM API skills.

Before StrataKV goes on the resume

- [ ] All 20 quiz questions answered out loud without notes
- [ ] All 10 learning-guide snippets explained against the real code
- [ ] Tests, fuzzer, and benchmarks run personally from a fresh clone
- [ ] Every resume number traced to a line in `docs/NUMBERS.md`
- [ ] A 60-second walkthrough ready: what it is, how crash safety is proven, one trade-off you would change with more time